

PUNCAK NIAGA · LIBRARY PORTAL

Technical Documentation

System Architecture, Data Model & Operations

Version 1.0 · Generated 19 May 2026
Laravel 10 · PHP 8.1+ · Tailwind CSS · Alpine.js

Contents

1. System Overview
2. Tech Stack
3. Project Structure
4. Database Schema
5. Domain Models
6. Controllers & Routing
7. Authentication & Authorisation
8. Frontend Architecture
9. Storage & File Handling
10. Setup & Deployment
11. Operational Tasks
12. Known Conventions & Gotchas

1. System Overview

The Library Portal is a digital media management platform that lets staff curate and publish a collection of photographs, videos and books (PDFs / Office docs), surface them to authenticated viewers through a fast browse-and-search interface, and group items into named collections. It also includes a personal "My List" feature, a VR tour module, and an administrative back-office for users, categories, tags, and a hero slideshow.

Primary actors

Role	Capabilities
Admin	Full CRUD on media, collections, categories, tags, slideshow and users. Sees all draft/published/archived rows.
User	Browse published media, view details, save items + collections to a personal "My List", change own password.
Guest	Land on the marketing home page and reach the login screen. No media access.

2. Tech Stack

Layer	Technology
Language	PHP 8.1+
Framework	Laravel 10.x
Database	MySQL 5.7+ (or MariaDB equivalent)
Frontend	Server-rendered Blade + Tailwind CSS (CDN) + Alpine.js

Auth	Laravel's session/cookie auth (laravel/ui scaffolding)
File storage	Local <code>storage/app/public</code> with <code>php artisan storage:link</code>
Notable packages	livewire/livewire, doctrine/dbal, barryvdh/laravel-dompdf, james-heinrich/getid3

3. Project Structure

```

app/
├─ Http/
│   └─ Controllers/          - 11 controllers (Media, Collection, MyList, User,
Category, Tag, Slideshow, Profile, Home, Login, Vr)
│   └─ Middleware/          - RedirectIfAuthenticated, PreventBackHistory, ...
│   └─ Kernel.php           - middleware aliases
├─ Models/                  - Media, Collection, MediaDetail, Category, Tag,
Slideshow, User
└─ Providers/

database/
├─ migrations/              - schema (renamed assets→media, FK columns updated)
└─ seeders/AdminUserSeeder.php

resources/views/
├─ auth/login.blade.php
├─ layouts/app.blade.php    - single layout for every page
├─ home.blade.php          - guest/landing
├─ users/                  - media browse + show + create + edit + collections/* +
mylist/*
├─ user/                   - admin user management
├─ collections/            - (legacy, see users/collections)
├─ categories/index.blade.php
├─ tags/index.blade.php
├─ slideshow/index.blade.php
└─ profile/edit.blade.php

```

└ docs/ – PDF templates (this file)

routes/web.php – all routes (no API routes used)

4. Database Schema

users

Column	Type	Notes
id	bigint	PK
name	varchar	
username	varchar	unique, used for login
email	varchar	unique
password	varchar	bcrypt hash
role	varchar	admin user
remember_token, email_verified_at, timestamps		

media

Column	Type	Notes
id	bigint	PK
uuid	uuid	unique; route binding key
type	varchar	photo video ebook
status	varchar(20)	indexed; draft published archived
title	varchar	

file_path	varchar(nullable)	relative path under <code>storage/app/public</code>
file_url	varchar(nullable)	external URL for "Use link" mode
date	date(nullable)	optional event date
thumbnail_path	varchar(nullable)	
timestamps		

media_details

Flexible key/value metadata attached to a media row. Keys in use: `views`, `collection`, `location` (book only), and any custom ad-hoc keys via `$media->getDetail($key, $default)`.

Column	Type	Notes
id	bigint	PK
media_id	bigint	FK media, cascade delete
key	varchar	
value	text	
timestamps		

Pivot tables

Table	Columns
media_category	id, media_id, category_id
media_tag	id, media_id, tag_id
my_list_items	id, user_id, media_id, timestamps · unique(user_id, media_id)
my_list_collections	id, user_id, collection_id, timestamps · unique(user_id, collection_id)

Lookup tables

Table	Columns
categories	id, name, timestamps
tags	id, name, timestamps
collections	id, uuid (unique), name (unique), timestamps
slideshows	id, title, slideshow_pic, timestamps

Collection link via metadata. Collections do not use a pivot table. The `media_details` row with `key='collection'`, `value=<collection name>` tags a media item as belonging to that collection. The `Collection` model exposes a UUID for routing; the join still happens through `media_details.value`.

5. Domain Models

Media (app/Models/Media.php)

- **Constants:** `Media::STATUSES = ['draft', 'published', 'archived']`.
- **Route key:** `uuid` — Media URLs use `/media/{uuid}`.
- **Auto UUID:** `booted()` assigns `uuid` on create if not supplied.
- **Accessors:**
 - `$media->resource_url` — external link or storage upload, whichever is set.
 - `$media->thumbnail_url` — explicit thumbnail → photo's source → default logo.
 - `$media->has_real_thumbnail` — true when a non-fallback cover exists.
 - `$media->collection` — reads `details` `key='collection'`.
- **Scopes:**
 - `Media::published()` — restricts to `status='published'`.

- `Media::inCollection($name)` — restricts to a collection by name.
- **Helpers:** `getDetail($key, $default)` , `incrementViews()` .

Collection

- UUID route key, auto-generated on create.
- `name` is unique; renaming propagates to all `media_details` rows on update.

User

- `isAdmin()` — case-insensitive: matches `'admin'` , `'1'` or integer `1` .
- `myList()` — belongsToMany(Media) via `my_list_items` .
- `myListCollections()` — belongsToMany(Collection) via `my_list_collections` .
- `mediaUploaded()` — hasMany Media (uploader column `uploaded_by` , still present in fillable on Media table for future use).

6. Controllers & Routing

Public/Auth routes

Method & path	Name	Controller
GET /, /home	home	HomeController@index
GET /login	login	LoginController
POST /login		LoginController
POST /logout	logout	LoginController

Authenticated routes

Method & path	Name	Purpose
---------------	------	---------

GET /media	media.index	Filtered/paginated browse
GET /media/{media}	media.show	Single media + viewer + related items
GET /media/create, POST /media	media.create / .store	Admin only
GET /media/{media}/edit, PUT /media/{media}	media.edit / .update	Admin only
DELETE /media/{media}	media.destroy	Admin only
GET /collections (+ create/show/edit/update/destroy/download)	collections.*	UUID-bound routes
GET /my-list, /my-list/add, POST /my-list/sync	mylist.*	Per-user saved items
GET /users, ... (CRUD)	users.*	Admin only
GET /categories, /tags, /slideshow (+ CRUD)		Admin only
GET /profile, PUT /profile/password	profile.*	Own password change
GET /vr, /vr-test	vr.*	VR tour module

Admin enforcement

Every admin-only controller uses the same constructor pattern:

```
public function __construct()
{
    $this->middleware('auth');
    $this->middleware(function ($request, $next) {
        abort_unless(auth()->user()->isAdmin(), 403);
        return $next($request);
    });
}
```

For mixed controllers (e.g. `MediaController` where listing/showing are public-auth but create/edit are admin), the second middleware is scoped via `->only([...)]`.

7. Authentication & Authorisation

- Login is by `username` (not email).
- No public registration. Admins create users from `/users/create`; the initial admin is seeded by `AdminUserSeeder`.
- The `role` column is the single source of truth. `$user->isAdmin()` drives every authz check.
- Logout uses SweetAlert2 confirmation; delete actions for media + collections use a matching SweetAlert pattern.

Hardening. Throttling on `POST /login` is provided by Laravel's `AuthenticatesUsers` trait. Keep `APP_DEBUG=false` and `APP_ENV=production` in production. The `storage:link` symlink must exist for uploads to resolve.

8. Frontend Architecture

One Blade layout — `resources/views/layouts/app.blade.php` — wraps every page.

- **Tailwind** is loaded via the CDN script with an inline theme extension (`lib-navy` , `lib-sky` , `lib-light` custom colors).
- **Alpine.js** is loaded via CDN. Used for: collapsible admin sidebar (state persisted to `localStorage`), category/tag pill multi-select, source toggle on the add-media form, SweetAlert confirm wrappers.
- **SweetAlert2** powers all destructive confirmations.
- **Custom pagination view:**
`resources/views/vendor/pagination/library.blade.php` ; invoked via `$paginator-`

```
>links('vendor.pagination.library') .
```

- **Admin sidebar** is a fixed left column (collapsible `w-56` ↔ `w-16`) on `md+` , an offcanvas drawer on smaller screens.

9. Storage & File Handling

What	Where	Cap
Media files (uploaded)	<code>storage/app/public/media</code>	50 MB
Thumbnails	<code>storage/app/public/media/thumbnails</code>	1 MB
Slideshow images	<code>storage/app/public/slideshows</code>	2 MB
Collection ZIP exports	<code>storage/app</code> (temp, deleted after send)	—

Allowed file types are scoped by the chosen media type:

Media type	Allowed extensions
photo	jpg, jpeg, png, gif, webp, bmp, svg
video	mp4, webm, mov, avi, mkv, m4v
ebook	pdf, doc, docx, xls, xlsx

Files are deleted from disk when the corresponding media row is destroyed.

Collections that are deleted release their tag from `media_details` but leave the files intact.

10. Setup & Deployment

Local setup

```
git clone
cd library
composer install
cp .env.example .env
php artisan key:generate
# configure DB credentials in .env
php artisan migrate
php artisan db:seed --class=AdminUserSeeder
php artisan storage:link
php artisan serve
```

Default admin credentials

Field	Default	Override
Username	admin	SEED_ADMIN_USERNAME
Email	admin@example.com	SEED_ADMIN_EMAIL
Password	Pnhb@12345	SEED_ADMIN_PASSWORD

Production checklist:

- Change the default admin password immediately after first login.
- Set `APP_ENV=production` , `APP_DEBUG=false` in `.env` .
- Run `php artisan config:cache` , `route:cache` , `view:cache` .
- Ensure `storage/` and `bootstrap/cache/` are writable by the web server.
- Configure a real session store (Redis or DB) if running multiple app servers.
- Tune PHP `upload_max_filesize` and `post_max_size` > 50 MB for media uploads.

11. Operational Tasks

Task	Command
Migrate fresh	<code>php artisan migrate:fresh --seed</code>
Clear caches	<code>php artisan optimize:clear</code>
Re-seed only the admin row	<code>php artisan db:seed --class=AdminUserSeeder</code>
Regenerate storage symlink	<code>php artisan storage:link</code>
Regenerate these PDFs	<code>php artisan docs:generate</code>

12. Known Conventions & Gotchas

- **UUID route binding:** `Media` and `Collection` resolve via `uuid`, not `id`. Always pass the model instance to `route()`, not the integer id.
- **Status visibility:** Non-admins never see `draft` or `archived` rows in any listing or related-items panel.
- **Related items:** A media item must share every category AND every tag of the current item to appear in Related Items. If either set is empty on the current item, the section is hidden.
- **Categories & tags:** Cannot be deleted while still attached to any media — the controller refuses and the UI greys out the Delete button.
- **Collections deletion:** Removes the `media_details` rows linking media to the collection, then deletes the collection. The media files themselves are not touched.
- **Sidebar state:** Collapsed/expanded saved to `localStorage` key `adminSidebarCollapsed`.
- **View persistence:** Grid/list toggle on `/media` stored in cookie `media_view` for 30 days.